- *Token filters*—Perform further processing on tokens produced by tokenizers. For example, token filters can change case, create synonyms, provide root words (stem), produce n-grams and shingles, and so on. Token filters are optional. There can be either zero, one, or many associated with an analyzer module. Elasticsearch provides a long list of token filters out of the box.

Character and token filters are optional, but we *must* have one tokenizer. We look at these components in detail later in the chapter, but first let's find an API to help test the analyzers before we put them into production.

7.2.4 Testing analyzers

You may be curious how Elasticsearch breaks text, modifies it, and then plays with it. After all, knowing how the text is split and enhanced up front helps us to choose the appropriate analyzer and customize it if needed.

Elasticsearch exposes an endpoint just for testing the text analysis process: the `_analyze` endpoint helps us understand the process in detail. This handy API allows us to test how the engine treats text during indexing. It's easy to understand with an example.

> **The _analyze endpoint**
>
> The `_analyze` endpoint helps a lot in understanding how text is treated and indexed by the engine, as well as why a search query may not produce the desired output. We can use this as the first step in testing our text with the expected analyzers before putting the code into production.

Let's say we want to determine how Elasticsearch will deal with this piece of text when it's indexed: "James Bond 007".

Listing 7.1 Testing the analyzer using the `_analyze` endpoint

```
GET _analyze
{
  "text": "James Bond 007"
}
```

Executing this script produces the set of tokens shown in figure 7.3. The output of the query shows us how the analyzer treats the text field. In this case, the field is split into three tokens (`james`, `bond`, and `007`), all lowercase. Because we didn't specify an analyzer in the code, by default, it is assumed to be the `standard` analyzer. Each token has a type: `ALPHANUM` for a string, `NUM` for a numeric token, and so on. The token's position is also saved, as you can see in the results. That brings us to the next point: specifying the analyzer explicitly during the `_analyze` test.